

CCE1 EXAMINATION QUESTION BANK — MASTER MODEL THEORY SOLUTIONS

Course Pattern: TE IT - 2024 Pattern (SPPU)

Academic Term: Semester V (2025-26 / 2026-27)

Document Type: In-Depth SPPU Theory Exam Model
Course Code: PCC-301-ITT / PCC-301-IT
Answers

IMPORTANT:

SPPU Model Theory Answer Standard: This master compilation provides comprehensive, exam-ready answers formulated specifically to satisfy SPPU 5-mark, 7-mark, and 10-mark evaluation schemes. All 28 questions from the CCE1 Question Bank across Unit 1 and Unit 2 are consolidated into **15 exhaustive topic solutions** featuring point-wise architectural breakdowns, parameter-based comparative tables, labeled system diagrams, shell scripts, POSIX C implementations, and step-by-step mathematical Gantt chart traces.

Unit 1: Overview of Operating System & System Structures (CO301.1)

Topic 1.1: Operating System Definition, Basic Functions, Objectives & Services

GROUP:

Consolidated Questions Covered:

- **Q1:** Define Operating System. List its basic functions.
- **Q2:** What are the objectives of an Operating System?
- **Q5:** State and explain different services provided by an operating system.

1. Academic Definition of an Operating System: An **Operating System (OS)** is system software that manages computer hardware and software resources, provides common services for computer programs, and acts as an intermediary interface between the computer hardware and the application user. From an architectural perspective, the OS can be viewed through three distinct paradigms:

- **Top-Down View (Extended Machine / Abstraction):** The OS abstracts away the raw, complex, and heterogeneous machine hardware (registers, disk sectors, hardware interrupts) into clean, high-level logical abstractions such as files, directories, processes, and virtual memory.
- **Bottom-Up View (Resource Manager / Allocator):** The OS acts as an omniscient resource manager that orchestrates, allocates, and deallocates hardware resources (CPU compute cycles, physical RAM, secondary storage blocks, I/O devices) among multiple competing user processes to maximize throughput and prevent conflicts.
- **Control Program View:** The OS operates as a supervisory control program that governs the execution of user programs to prevent system errors, deadlocks, and unauthorized access to protected resources.

2. Primary Objectives of an Operating System:

1. **Convenience (User-Friendliness):** Hides low-level hardware intricacies and instruction sets, providing users and software developers with an intuitive graphical or command-line interface and standard Application Programming Interfaces (APIs).
2. **Efficiency (Optimal Resource Utilization):** Ensures that the underlying CPU, memory, and peripheral hardware are utilized at maximum possible capacity with minimal idle time through advanced CPU scheduling, memory swapping, and asynchronous I/O spooling.

3. **Ability to Evolve (Extensibility & Modularity):** Engineered with modular and layered architectures so that new hardware features, updated peripheral device drivers, security patches, and system services can be seamlessly integrated without overhauling the entire system.
4. **Protection and Security:** Guarantees strict isolation between concurrent user processes and prevents unauthorized tampering with system kernel structures and user files.
- 3. Basic Functions & Core System Services:** Operating system functions and services are structured into **User-Oriented Services** (facilitating program development and execution) and **System-Oriented Services** (ensuring overall system efficiency and integrity):

OS Service / Function	Detailed Technical Description & Role in System
Program Execution	Loads executable binary images from secondary storage into main memory (RAM), initializes execution state, allocates stack/heap spaces, assigns CPU time, and handles normal termination or graceful abort on fatal errors.
I/O Operations Management	Abstracts complex hardware controller interactions via unified device drivers. Implements buffering, caching, and spooling to bridge the speed disparity between high-speed CPUs and slow peripheral devices.
File System Manipulation	Provides hierarchical file and directory structures. Manages creation, deletion, reading, writing, and searching of files, and translates logical byte offsets into physical disk sectors while enforcing access permissions.
Inter-Process Communication (IPC)	Enables concurrent processes to exchange information and synchronize execution on the same host (via shared memory, message queues, pipes) or across distributed networks (via network sockets).
Resource Allocation	Dynamically assigns CPU scheduling priorities, physical memory frames, and I/O channels among multiple active processes to ensure fairness, high throughput, and absence of starvation.
Error Detection & Handling	Continuously monitors all system layers for hardware faults (memory parity errors, disk bad sectors, power anomalies) and software exceptions (division by zero, null pointer dereferences, segmentation faults), taking corrective actions to maintain stability.
Accounting & System Monitoring	Logs and tracks cumulative resource consumption (CPU time, memory footprint, disk usage) per user or process for system auditing, billing, performance tuning, and capacity planning.
Protection and Security	Enforces dual-mode CPU protection (User vs. Kernel mode), authenticates user logins, controls access control lists (ACLs), and prevents rogue processes from corrupting system memory.

Source: Silberschatz et al., Ch 1 & 2, p. 3-65 | Source: Stallings, Ch 1, p. 50-75

Topic 1.2: Dual-Mode Operation: User Level vs. Kernel Level Execution

GROUP:

Consolidated Questions Covered:

- **Q4:** Differentiate between user level and kernel level.

1. Architectural Rationale & Need for Dual-Mode Protection: In a multiprogrammed operating system, multiple user programs execute concurrently. If a user application had direct, unrestricted access to the computer's physical hardware, a buggy or malicious program could execute instructions that halt the CPU (HLT), overwrite the kernel's memory space, or corrupt the disk file system. To prevent catastrophic system crashes and enforce process isolation, modern computer architectures implement hardware-level **Dual-Mode Operation**.

2. The Hardware Mode Bit & State Transition Mechanics:

- The CPU hardware maintains a dedicated register flag called the **Mode Bit** (typically within the Processor Status Word / EFLAGS register).
 - **Mode Bit = 1 (User Mode / Unprivileged Level):** The CPU executes ordinary user applications with restricted instruction capabilities and isolated memory space.
 - **Mode Bit = 0 (Kernel Mode / Supervisor / Privileged Level):** The CPU executes operating system core code with complete, unrestricted access to all hardware registers, memory addresses, and CPU control instructions.

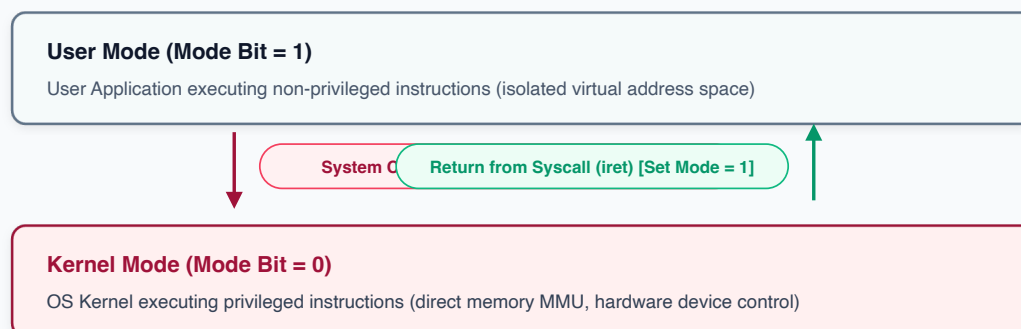


Figure 1.1: Dual-Mode Operation and System Call Transition

Step-by-Step Transition Mechanism:

1. **Trigger:** An application running in User Mode requires an OS service (e.g., reading a file via `read()`). It invokes a **System Call**.
2. **Trap Generation:** The system call executes a software trap / exception instruction (such as `syscall`, `sysenter`, or `INT 0x80`).
3. **Hardware Mode Switch:** The CPU hardware automatically switches the Mode Bit from 1 to 0 and transfers control to the predefined entry point in the Interrupt Vector Table (IVT).
4. **Kernel Execution:** The kernel verifies request parameters, checks permissions, and executes the privileged service in Kernel Mode.
5. **Return to User Space:** Upon completing the service, the kernel executes a privileged return instruction (`sysret` / `iret`). The hardware automatically resets the Mode Bit from 0 back to 1 and resumes user process execution.

3. Privileged vs. Non-Privileged Instructions:

- **Privileged Instructions (Kernel Mode Only):**
 - Halting the CPU (HLT), managing timer hardware to prevent CPU monopolization.
 - Modifying the Memory Management Unit (MMU) base registers (e.g., loading CR3 page table register in x86).
 - Enabling/disabling hardware interrupts (CLI / STI).
 - Direct I/O port read/write operations (IN / OUT).
- **Non-Privileged Instructions (Allowed in User Mode):**
 - Arithmetic and logical computations (ADD, SUB, AND, OR, MUL).
 - Register-to-register data movement and branching instructions within user address bounds.
 - Memory read/write within the process's allocated virtual address space.

4. Comprehensive Parameter-Based Comparison Table:

Parameter	User Level (User Mode)	Kernel Level (Kernel Mode)
Hardware Mode Bit	Mode Bit is set to 1.	Mode Bit is set to 0.
Privilege Level	Restricted / unprivileged privilege level (Ring 3 in x86 architecture).	Highest / supervisory privilege level (Ring 0 in x86 architecture).
Direct Hardware Access	Strictly prohibited; cannot issue raw I/O instructions or access physical hardware registers.	Unrestricted; has direct, low-level control over CPU timers, disk controllers, and physical devices.

Parameter	User Level (User Mode)	Kernel Level (Kernel Mode)
Instruction Execution	Can execute only non-privileged machine instructions.	Can execute both non-privileged and privileged instructions (HLT, CLI, LIDT).
Memory Protection	Confined strictly within the process's assigned virtual address space; accessing kernel RAM triggers a segfault.	Full read/write access to both physical RAM and kernel address space.
Fault Impact	A runtime fault (e.g., divide by zero) crashes only that isolated user process.	An unhandled fault causes an OS kernel panic, BSOD, or complete system freeze.
Transition Trigger	Transitions to Kernel Mode via System Calls, Software Traps, or Hardware Interrupts.	Transitions back to User Mode via sysret or iret instructions.
Representative Code	User applications, compilers, database engines, web browsers, shells.	OS kernel core, device drivers, scheduler, memory manager, interrupt handlers.

Source: Silberschatz et al., Ch 1, p. 20-28 | Source: Tanenbaum, Ch 1, p. 45-58

Topic 1.3: Shell Scripting & Practical Linux Command Utilities

GROUP:

Consolidated Questions Covered:

- **Q3:** Write a BASH shell script to find factorial of a number and explain constructs.
- **Q6:** Explain shell commands with examples: grep, sort, chmod, cat, echo.
- **Q11:** Write a shell script to check file or directory existence.

1. Factorial BASH Script with Comprehensive Construct Breakdown:

```
#!/bin/bash
# SPPU Model Script: Calculate Factorial of a Non-Negative Integer

echo -n "Enter a non-negative integer: "
read num

# Input Validation: Check if input is empty or contains non-numeric / negative values
if [ -z "$num" ] || ! [[ "$num" =~ ^[0-9]+$ ]]; then
    echo "Error: Invalid input! Please enter a valid non-negative integer."
    exit 1
fi

fact=1
temp=$num

# Iterative loop to calculate factorial
while [ $temp -gt 1 ]; do
    fact=$((fact * temp))
    temp=$((temp - 1))
done

echo "====="
echo "Result: The Factorial of $num ($num!) = $fact"
echo "====="
exit 0
```

Detailed Explanation of Shell Constructs Used:

- **#!/bin/bash (Shebang):** Absolute path specifying that the BASH shell interpreter must be used to parse and execute this script.

- `read num`: Captures user standard input from the terminal and stores it dynamically in the variable `num`.
- `if [ ... ]; then ... else ... fi`: Conditional branching construct that evaluates test conditions and executes appropriate blocks based on the return status (0 for true, non-zero for false).
- `[ -z "$num" ]`: Test operator checking if the input string is empty (length zero).
- `[[ "$num" =~ ^[0-9]+$ ]]`: Extended regex matching ensuring only unsigned numeric digits are accepted.
- `while [ $temp -gt 1 ]; do ... done`: Iterative looping construct that continues execution as long as the condition `$temp -gt 1` evaluates to true.
- `$( ( expression ) )`: Built-in BASH arithmetic expansion mechanism used to perform fast integer arithmetic without invoking external processes like `expr` or `bc`.
- `exit 0` / `exit 1`: Returns an exit status code to the parent shell indicating successful execution (0) or runtime error (1).

2. Essential Shell Commands Explained with Syntax, Options, and Examples:

• `grep` (Global Regular Expression Print):

- **Function:** Scans files or standard input line-by-line for text matching a regular expression pattern and outputs matching lines.
- **Syntax:** `grep [options] "pattern" [file_path]`
- **Key Options:** `-i` (case-insensitive), `-n` (show line numbers), `-v` (invert match / show non-matching lines), `-c` (count matching lines), `-r` (recursive directory search).
- **Exam Example:**

```
grep -i -n "error" /var/log/syslog
# Searches for 'error' (ignoring case) in syslog and prints matching lines with line numbers.
```

• `sort` (Sort Lines of Text Files):

- **Function:** Orders lines of text files alphabetically, numerically, or according to user-defined key columns.
- **Syntax:** `sort [options] [file_path]`
- **Key Options:** `-n` (numerical sort), `-r` (reverse sort order), `-k N` (sort on key column *N*), `-u` (suppress duplicate lines / unique only).
- **Exam Example:**

```
sort -n -r scores.txt
# Numerically sorts the numeric values in scores.txt in descending order.
```

• `chmod` (Change File Access Permissions / Mode):

- **Function:** Modifies filesystem read (`r=4`), write (`w=2`), and execute (`x=1`) permissions for Owner, Group, and Others.
- **Syntax:** `chmod [octal_mode] [file_path]` OR `chmod [references][operator][modes] [file_path]`
- **Exam Example:**

```
chmod 754 deploy.sh
# Sets Owner = 7 (rwx), Group = 5 (r-x), Others = 4 (r--) on deploy.sh.
chmod u+x,g-w script.sh
# Adds execute permission to owner and removes write permission from group.
```

• `cat` (Concatenate and Display File Content):

- **Function:** Reads one or more files sequentially and writes their contents to standard output.
- **Syntax:** `cat [options] [file1] [file2]`
- **Key Options:** `-n` (number all output lines), `-b` (number non-blank lines), `-s` (squeeze multiple adjacent blank lines).
- **Exam Example:**

```
cat file1.txt file2.txt > combined.txt
# Concatenates file1.txt and file2.txt into combined.txt.
```

• `echo` (Print Line of Text / Variables to Output):

- **Function:** Writes strings, expanded variables, and escape-formatted text to standard output.

- **Syntax:** `echo [options] [string/variable]`
- **Key Options:** `-n` (do not output trailing newline), `-e` (enable interpretation of backslash escape sequences like `\n`, `\t`).
- **Exam Example:**

```
echo -e "User: $USER\nHome Directory: $HOME"
# Prints username and home path separated by a newline.
```

3. File / Directory Existence Check Script:

```
#!/bin/bash
# SPPU Model Script: Check Existence and Type of File or Directory

echo -n "Enter the file or directory path to check: "
read target_path

# Verify if path exists
if [ ! -e "$target_path" ]; then
    echo "Result: The path '$target_path' DOES NOT exist."
    exit 1
fi

echo "Result: The path '$target_path' EXISTS."

# Determine specific file type and permissions
if [ -d "$target_path" ]; then
    echo "Type: It is a DIRECTORY."
    echo "Directory contents count: $(ls -l "$target_path" | wc -l) items."
elif [ -f "$target_path" ]; then
    echo "Type: It is a REGULAR FILE."
    echo "File Size: $(ls -lh "$target_path" | awk '{print $5}')"
    # Check permissions
    [ -r "$target_path" ] && echo "- Read permission: Granted" || echo "- Read permission: Denied"
    [ -w "$target_path" ] && echo "- Write permission: Granted" || echo "- Write permission: Denied"
    [ -x "$target_path" ] && echo "- Execute permission: Granted" || echo "- Execute permission: Denied"
else
    echo "Type: It is a SPECIAL FILE (socket, pipe, device node, or symlink)."
fi
exit 0
```

Summary of Linux File Test Operators for SPPU Theory:

- `-e` file: True if file exists (regardless of type).
- `-f` file: True if file exists and is a regular file.
- `-d` file: True if file exists and is a directory.
- `-r` file / `-w` file / `-x` file: True if readable / writable / executable.
- `-s` file: True if file exists and has size greater than zero bytes.

Source: Advanced BASH Scripting Guide, Ch 4-7 | Source: Stallings, Ch 2, p. 80-95

Topic 1.4: Operating System Structure & Design (Monolithic, Microkernel, Layered, Modular)

GROUP:

Consolidated Questions Covered:

- **Q9:** Analyze monolithic and microkernel architectures with respect to structure and performance.
- **Q10:** Compare monolithic and microkernel architectures.
- **Q12:** Discuss layered and modular approach to OS design.

1. Monolithic Kernel Architecture: In a **Monolithic Operating System**, the entire operating system codebase—including CPU scheduling, memory management, file systems, device drivers, and network protocol stacks—is compiled together and executes as a single massive binary in a single unified address space inside **Kernel Space**.

- **Execution Mechanism:** Components communicate with each other via fast, direct C function calls without requiring message passing or context switches.
- **Advantages:** High throughput and raw performance; minimal overhead because all kernel routines share data structures directly in memory.
- **Disadvantages:** Poor fault isolation; since everything shares the same address space, a bug or null pointer dereference in any device driver can corrupt kernel structures and trigger a total system crash. Maintenance and updates are difficult.
- **Examples:** Linux, FreeBSD, OpenBSD, Traditional UNIX, MS-DOS.

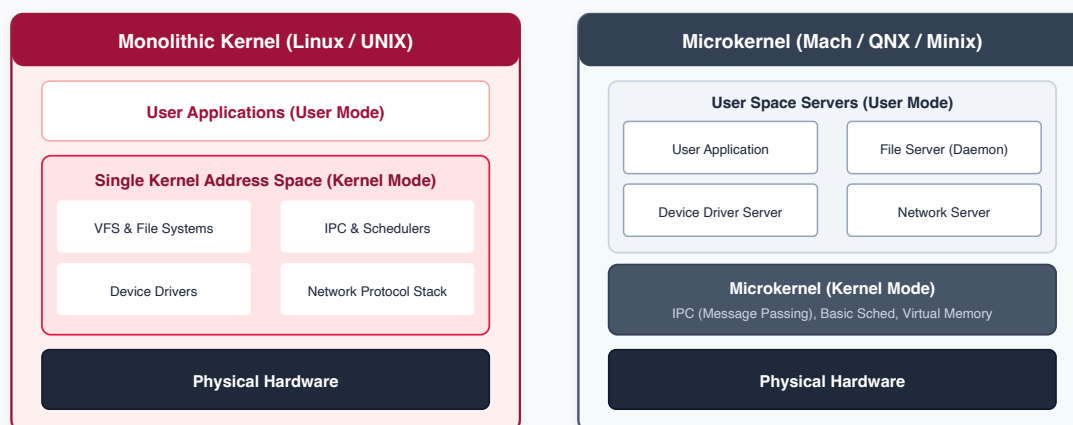


Figure 1.2: Monolithic Kernel vs. Microkernel Architecture

2. Microkernel Architecture: A **Microkernel** strips down the kernel to bare minimum essentials: low-level address space management, thread scheduling, and Inter-Process Communication (IPC). All other non-essential operating system services (File Systems, Device Drivers, Network Stacks, Virtual Memory Managers) are removed from kernel space and run as independent, unprivileged server processes in **User Space**.

- **Execution Mechanism:** User applications (clients) and OS services (servers) communicate by exchanging discrete IPC messages mediated by the small microkernel.
- **Advantages:** Superior fault isolation, reliability, and security. If a file system or network driver crashes, it can be restarted in user space without bringing down the operating system. Highly extensible.
- **Disadvantages:** Performance overhead due to frequent context switching and message serialization across user-kernel boundaries for every system interaction.
- **Examples:** Mach, QNX Neutrino, Minix 3, seL4, Symbian OS.

3. Parameter-Based Comparison: Monolithic vs. Microkernel:

Parameter	Monolithic Kernel	Microkernel
Kernel Size & Footprint	Large; entire operating system is compiled into a multi-megabyte kernel binary image.	Minimal; core kernel is very small (often under 50,000 lines of code).
Service Location	All services (VFS, IPC, memory, drivers, networking) reside in Kernel Space.	Only core IPC, basic scheduling, and hardware primitives reside in Kernel Space; rest run in User Space.
Communication Model	Direct internal function calls sharing kernel memory data structures.	Message passing (IPC) mediated by the microkernel.
Execution Performance	Very high throughput; zero IPC message passing overhead.	Lower performance due to frequent context switches and IPC copying overhead.

Parameter	Monolithic Kernel	Microkernel
Fault Isolation & Security	Poor; a fatal bug in a third-party driver crashes the whole operating system.	Excellent; buggy services crash only their user-space daemon and can be restarted dynamically.
Extensibility	Complex; adding features requires kernel recompilation or loadable module insertion.	Simple; new services are added simply by starting new user-space server processes.
Debugging & Testing	Difficult; debugging kernel-space code requires hardware debuggers or serial logs.	Easy; services run as regular user processes and can be debugged with standard tools (gdb).
Representative Systems	Linux, FreeBSD, Solaris, MS-DOS, macOS XNU (hybrid core).	Mach, QNX, Minix 3, seL4, L4 microkernel family.

4. Layered and Modular Approaches to Operating System Design:

• Layered Approach (Dijkstra's THE System):

- **Design Principle:** The OS is broken down into a hierarchy of N distinct layers (0 to N). Layer 0 represents the raw Hardware, while Layer N represents the User Interface / Shell. Each intermediate layer is implemented strictly using the services and operations provided only by lower layers ($0 \dots i - 1$).
- **Advantages:** High modularity and simplified debugging. Verification is bottom-up: once Layer 0 and Layer 1 are tested and verified, Layer 2 can be debugged with certainty that lower layers are error-free.
- **Disadvantages:** Strict layering is difficult to define in practice (e.g., storage driver needs memory management buffering, but virtual memory requires disk backing storage). Inefficient runtime overhead due to function call pass-through across multiple layers.

• Modular Approach (Loadable Kernel Modules - LKM):

- **Design Principle:** The core kernel provides fundamental scheduling and memory management, while dynamic modules (such as device drivers, file system handlers like ext4/NTFS, and network protocols) are loaded and linked into kernel space on-demand at runtime without rebooting.
- **Advantages:** Combines the raw execution speed of Monolithic systems (modules run inside kernel space with direct function calls) with the flexibility and extensibility of Microkernels.
- **Modern Implementations:** Modern Linux (insmod, modprobe, rmmod), macOS kernel extensions, Oracle Solaris.

Source: Silberschatz et al., Ch 2, p. 70-85 | Source: Tanenbaum, Ch 1, p. 60-78

Topic 1.5: Symmetric Multiprocessing (SMP), Virtual Machines & OS Performance Evaluation

GROUP:

Consolidated Questions Covered:

- **Q7:** Explain the concept of virtual machine with its benefits.
- **Q8:** Demonstrate application of SMP; illustrate concurrent execution and benefits over uniprocessors.
- **Q13:** Evaluate different operating systems based on performance.

1. Symmetric Multiprocessing (SMP):

- **Architecture:** In an SMP system, two or more identical physical CPU cores are connected to a single, shared main memory (RAM) and shared I/O bus, all managed symmetrically by a single instance of the operating system.
- **Comparison with Asymmetric Multiprocessing (AMP / Master-Slave):**
 - **AMP:** One master processor runs OS kernel code and assigns tasks to slave processors (creates an architectural bottleneck).
 - **SMP:** Any processor can execute OS kernel code and user tasks concurrently. The kernel must be reentrant and employ fine-grained locking (mutexes/spinlocks) to prevent race conditions on shared kernel data structures.
- **Benefits over Uniprocessor Systems:**

1. **Increased Throughput:** More processes and threads complete execution per unit time through true parallel computation.
2. **Economy of Scale:** Multiprocessors share power supplies, memory buses, motherboards, and storage arrays, resulting in lower cost per unit compute compared to multiple distinct computers.
3. **Increased Reliability & Graceful Degradation (Fault Tolerance):** If one CPU core experiences a hardware failure, the system does not crash; instead, the OS disables the faulty core and continues execution at slightly reduced throughput (fail-soft operation).

2. Virtual Machine Concepts & Benefits: A **Virtual Machine (VM)** is an isolated software abstraction of a complete computer system created and managed by a software virtualization layer called the **Hypervisor** or **Virtual Machine Monitor (VMM)**.

- **Types of Hypervisors:**

- **Type-1 (Bare-Metal) Hypervisors:** Run directly on the physical hardware without an underlying host OS (e.g., VMware ESXi, Xen, Linux KVM). Delivers high performance.
- **Type-2 (Hosted) Hypervisors:** Run as an application process on top of a standard host OS (e.g., Oracle VirtualBox, VMware Workstation).

- **Core Benefits in Enterprise Computing:**

- **Server Consolidation:** Enables running multiple disparate OS instances (Linux, Windows Server) concurrently on a single physical host, slashing hardware footprint, power, and cooling costs.
- **Security & Sandboxing:** Complete guest OS isolation ensures that security breaches or crashes in one VM cannot affect other VMs or the physical host.
- **Rapid Provisioning & Disaster Recovery:** VMs exist as file images, enabling instant snapshotting, cloning, and live migration across data centers.
- **Development & Testing:** Allows developers to simulate complex multi-tier network topologies and multiple OS platforms on a single workstation.

3. Performance Evaluation Criteria across Operating Systems: To objectively evaluate and benchmark operating systems for specific enterprise workloads, several key metrics and evaluation methodologies are used:

- **Primary Evaluation Metrics:**

- **CPU Utilization:** Percentage of active compute time spent executing useful user code versus kernel overhead and idle time (target: 40% to 90%).
- **Throughput:** Number of processes completed per unit time (e.g., transactions/sec).
- **Turnaround Time:** Total elapsed time from process submission to complete termination.
- **Waiting Time:** Cumulative time a process spends waiting in the Ready Queue.
- **Response Time:** Time elapsed from job submission until the first output response is produced (critical in interactive and real-time systems).
- **Scalability & Multi-core Contention:** Ability of the OS to maintain linear performance scaling from 4 to 256 CPU cores without lock contention bottlenecks in the scheduler or memory allocator.

- **Performance Evaluation Methodologies:**

1. **Deterministic Modeling:** Takes a predetermined analytical workload and calculates exact mathematical performance metrics (Gantt chart analysis).
2. **Queuing Network Models:** Models CPU queues and I/O channels mathematically using Little's Formula ($N = \lambda \times W$).
3. **Simulation:** Simulates OS hardware interactions using probabilistic event-driven software models.
4. **Direct Implementation & Benchmarking:** Executes standardized real-world benchmark suites (e.g., SPEC CPU, UnixBench, Geekbench) on production hardware.

Source: Stallings, Ch 1 & 2, p. 45-78 | Source: Silberschatz et al., Ch 2 & 16, p. 80-110, 680-710

Topic 1.6: Linux Booting Process

GROUP:

Consolidated Questions Covered:

- **Q14:** Explain Linux booting process in detail.

Step-by-Step Chronological Linux Boot Sequence: The Linux booting architecture transitions through six distinct, sequentially linked phases from power-on to user login:

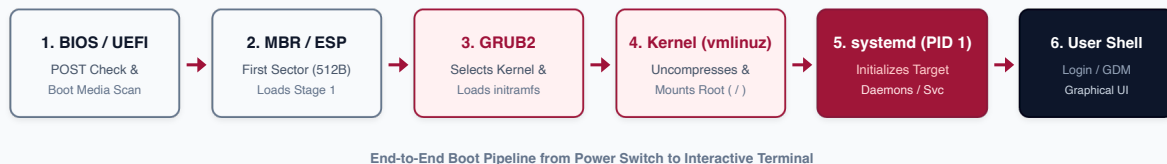


Figure 1.3: Chronological Linux Boot Sequence

1. BIOS / UEFI Phase (Hardware Initialization & POST):

- When power is switched on, the CPU initializes its program counter to execute the firmware residing in ROM/Flash memory (BIOS or modern UEFI).
- Executes the **Power-On Self-Test (POST)** to verify hardware integrity (RAM, CPU registers, storage controllers, keyboard).
- Reads boot device sequence from CMOS RAM, searches the designated boot device for the Master Boot Record (MBR - Sector 0, 512 bytes: 446 bytes boot code, 64 bytes partition table, 2 bytes magic signature 0x55AA) or UEFI EFI System Partition (ESP), loads the initial bootloader code into RAM, and passes control to it.

2. Bootloader Phase (GRUB / GRUB2):

- **Stage 1:** Primary bootloader code in MBR points to Stage 1.5/Stage 2.
- **Stage 2:** GRUB2 loads filesystem drivers to read `/boot/grub/grub.cfg`, presents the OS selection menu to the user, and loads the selected compressed Linux kernel binary (`vmlinuz`) and the initial RAM disk image (`initramfs` / `initrd`) into physical main memory.

3. Kernel Initialization Phase:

- The kernel uncompresses itself in RAM and performs low-level hardware probing and CPU subsystem initialization (MMU, page tables, interrupt vectors).
- Mounts the `initramfs` (temporary root filesystem in RAM) to load essential hardware storage drivers (e.g., NVMe, SATA, RAID controllers).
- Once disk drivers are loaded, the kernel unmounts `initramfs` and mounts the true root filesystem (`/`) in read-only mode to perform filesystem integrity checks (`fsck`).
- Remounts `/` in read-write mode and executes the first user-space process: `/sbin/init` or `/lib/systemd/systemd`.

4. Init / Systemd Phase (PID 1 - Parent of All Processes):

- The Linux kernel spawns `systemd` (or legacy `SysVinit`) as **PID 1**, the ancestor of all subsequent processes in user space.
- `Systemd` reads `/etc/systemd/system/default.target` to determine the target state (e.g., `multi-user.target` for text mode or `graphical.target` for GUI mode, superseding legacy `SysVinit` runlevels 0 to 6).
- Resolves dependency graphs and launches system targets concurrently in parallel.

5. System Daemons & Services Initialization:

- `Systemd` initializes background system daemons: `systemd-journald` (logging), `NetworkManager` (network interfaces), `sshd` (secure shell), `cron` (job scheduling), and `D-Bus` (inter-process message bus).

6. Display Manager / Login Shell Phase:

- Launches virtual terminal consoles (`getty` / `agetty` on `tty1` to `tty6`) for CLI login, or starts the Display Manager (`gdm`, `lightdm`, `sddm`) for graphical GUI login.

- The user authenticates via PAM (Pluggable Authentication Modules). Upon verification, the system spawns the user's default login shell (/bin/bash) or desktop session (GNOME, KDE).

Source: Silberschatz et al., Ch 2, p. 88-95 | Source: Stallings, Ch 2, p. 90-105

Unit 2: Process & Thread Management, CPU Scheduling (CO301.2)

Topic 2.1: Process Concept, Five-State Process Model & Process Control Block (PCB)

GROUP:

Consolidated Questions Covered:

- **Q1:** Draw and explain the five state process diagram.
- **Q5:** Explain with suitable diagram how fields in PCB are used to maintain process information.
- **Q7:** Draw and explain process state diagram.

1. Academic Concept of a Process & Memory Architecture: A **Process** is an active program in execution. While a program is a passive entity stored on disk as an executable binary file (e.g., a.out), a process is an active dynamic entity that includes the program code along with dynamic resources, execution state, and memory allocations.

• Memory Layout of a Process:

- **Text (Code) Segment:** Contains executable machine instructions; read-only and shared among concurrent instances.
- **Data Segment:** Contains initialized global and static variables.
- **BSS Segment:** Contains uninitialized global and static variables (zero-initialized by OS).
- **Heap Segment:** Dynamically allocated runtime memory (via `malloc()` / `new`); grows dynamically upward toward higher memory addresses.
- **Stack Segment:** Stores temporary function call frames, local variables, and return addresses; grows downward toward lower memory addresses.

2. Five-State Process Lifecycle Model: During its active execution lifetime, a process transitions through five distinct states governed by the operating system:

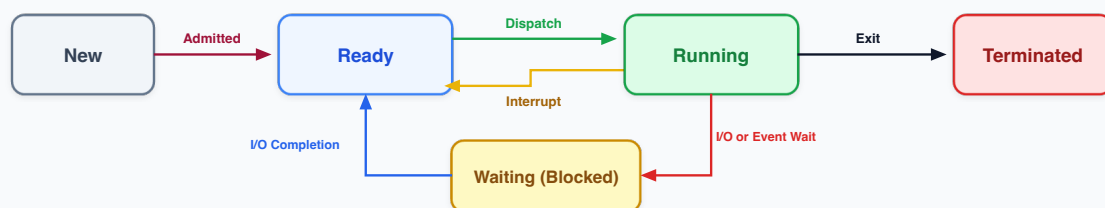


Figure 2.1: Five-State Process Lifecycle State Transitions

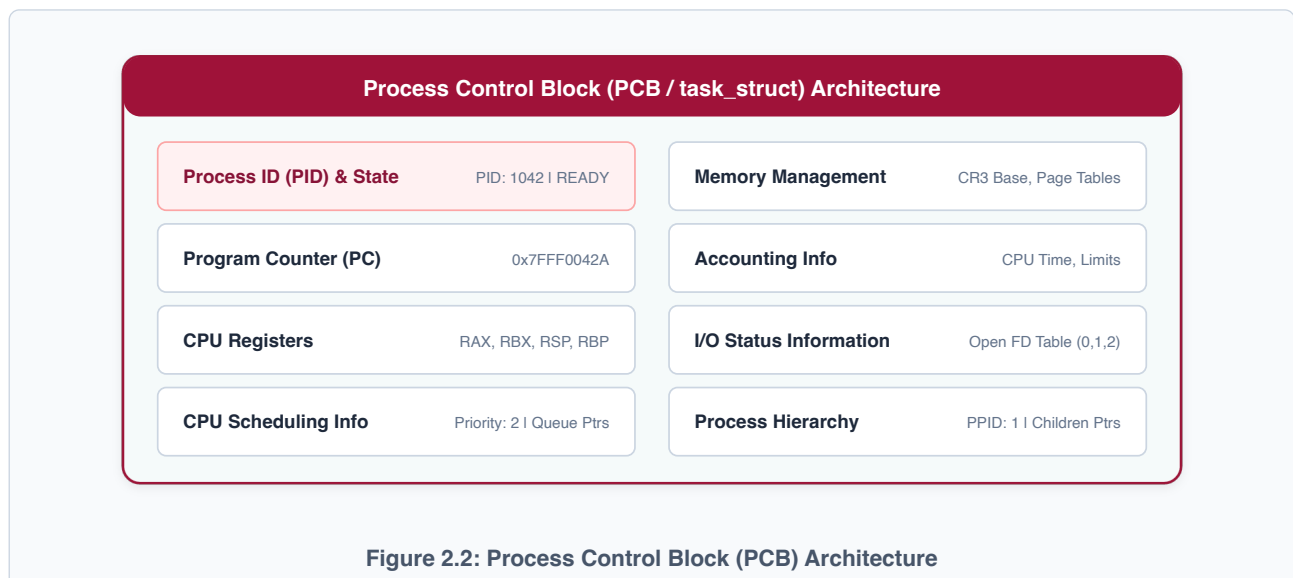
- **1. New (Created):** The process is in the initial stage of creation; its program code is being loaded and its PCB is being allocated by the Long-Term Scheduler (Job Scheduler).
- **2. Ready:** The process is resident in main memory (RAM) and waiting in the Ready Queue to be assigned a CPU core by the Short-Term Scheduler.
- **3. Running:** The process has been dispatched by the CPU scheduler; its instructions are actively executing on a physical CPU core.
- **4. Waiting (Blocked):** The process cannot execute because it is waiting for an asynchronous event (e.g., completion of disk I/O, user keyboard input, reception of a network packet, or acquiring a mutex lock).

- **5. Terminated (Exit):** The process has finished executing its final statement or has been terminated via `exit()`; the OS deallocates its memory and reclaims its resources while keeping exit status for the parent.

State Transition Triggers:

- **Admitted (New → Ready):** Long-Term Scheduler admits the process into main memory ready queue.
- **Scheduler Dispatch (Ready → Running):** Short-Term Scheduler assigns CPU to the top ready process.
- **Interrupt / Time Quantum Expiry (Running → Ready):** Preempted by timer interrupt in Round Robin scheduling or by higher-priority process arrival.
- **I/O or Event Wait (Running → Waiting):** Process issues a blocking system call (e.g., `read()`).
- **I/O or Event Completion (Waiting → Ready):** Hardware raises interrupt indicating I/O completion; OS moves process back to Ready Queue.
- **Exit (Running → Terminated):** Process completes execution or encounters a fatal exception.

3. Process Control Block (PCB) Structure & Detailed Field Analysis: The OS maintains a dedicated kernel data structure called the **Process Control Block (PCB)** (represented as `struct task_struct` in Linux) for every active process:



Detailed Breakdown of All PCB Fields for SPPU Theory:

1. **Process Identifier (PID & PPID):** A unique non-negative integer identifying the process within the system, along with the PID of its parent process.
2. **Process State:** The current operational state of the process (New, Ready, Running, Waiting, Terminated).
3. **Program Counter (PC):** A 32-bit or 64-bit hardware register address indicating the memory location of the next machine instruction to be fetched and executed.
4. **CPU Hardware Registers:** Contains the state of all CPU registers (Accumulator, Data Registers, Base Register, Index Registers, Stack Pointer, and Condition Code flags) saved during context switches.
5. **CPU Scheduling Information:** Stores process scheduling priority, pointers to scheduling queues (Ready queue, Device queue), and scheduling algorithm parameters.
6. **Memory Management Information:** Contains page table pointers, segment table descriptors, base and limit registers, and virtual memory maps used by the MMU to translate virtual addresses.
7. **Accounting & Auditing Information:** Tracks cumulative CPU compute time consumed, clock ticks, real-time elapsed, time limits, and account IDs for billing and monitoring.
8. **I/O Status Information:** Maintains a table of open file descriptors (fd table), allocated peripheral I/O devices, and outstanding I/O transfer requests.

Source: Silberschatz et al., Ch 3, p. 105-115 | Source: Stallings, Ch 3, p. 110-135

Topic 2.2: Context Switching Mechanism & System Performance Impact

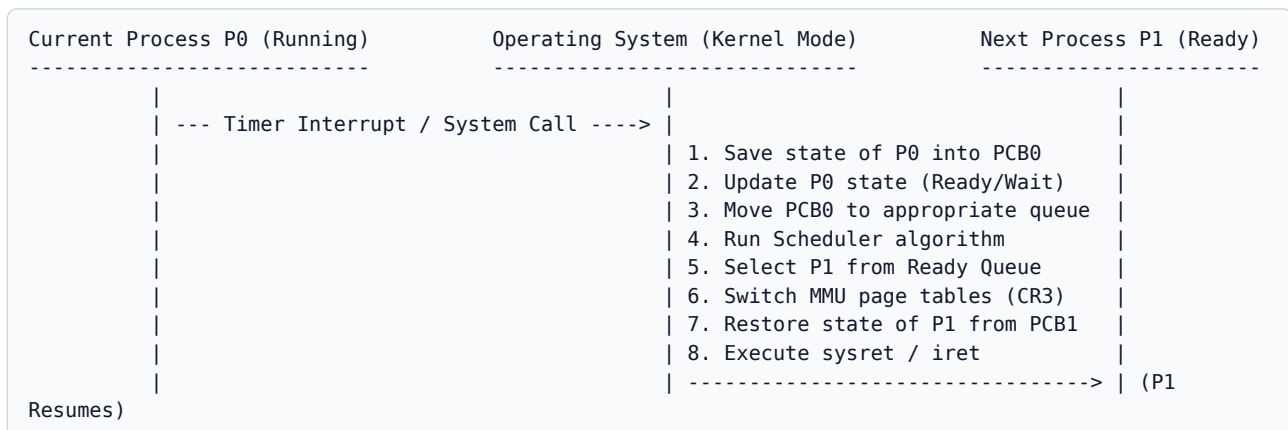
GROUP:

Consolidated Questions Covered:

- **Q4:** Analyze the context-switching mechanism and its impact on process execution and system performance.
- **Q11:** Analyze context-switching mechanism and its impact on performance.

1. Academic Definition of Context Switching: A **Context Switch** is the low-level operating system mechanism that suspends the execution of an active process on a CPU core, saves its entire hardware state (context) into its PCB, and restores the saved execution context of another ready process from its PCB to resume execution. Context switching enables multitasking and time-sharing in modern operating systems.

2. Detailed Chronological Steps in Context Switching: When a hardware timer interrupt, I/O trap, or higher-priority preemption occurs, the OS executes the following strict sequence:



1. **State Preservation of Current Process (P_0):** The CPU hardware switches to Kernel Mode. The kernel saves the Program Counter, general-purpose registers, stack pointers, and condition flags into P_0 's PCB.
2. **State Update & Queue Placement:** The OS updates P_0 's state from Running to Ready (if preempted) or Waiting (if blocked on I/O) and places its PCB into the corresponding queue.
3. **Scheduler Invocation:** The Short-Term CPU Scheduler selects the next candidate process (P_1) based on the active CPU scheduling policy (e.g., Round Robin, SJF, Priority).
4. **Memory Context Switching:** The OS reloads the memory management unit base register (e.g., reloading the CR3 page directory pointer register in x86), establishing P_1 's virtual address space.
5. **State Restoration of Selected Process (P_1):** The kernel loads the hardware register values, stack pointers, and Program Counter saved in P_1 's PCB into the physical CPU registers.
6. **Execution Resumption:** The kernel executes `iret` / `sysret`, switching CPU hardware back to User Mode (Mode Bit = 1). P_1 resumes instruction execution seamlessly from the exact instruction address where it was previously suspended.

3. Performance Overhead & System Impact Analysis:

Context switching is **pure computational overhead** because the CPU executes no productive user application work during the switch. The impact on system performance encompasses three major factors:

- **1. Direct Computational Latency:**
 - Saving and loading dozens of 64-bit hardware registers, manipulating scheduling queues, and executing scheduler code takes between 1 to 10 microseconds per context switch. If a system performs thousands of context switches per second, a substantial percentage of total CPU cycles is lost to switching overhead.
- **2. Translation Lookaside Buffer (TLB) Invalidation:**
 - Switching address spaces between distinct processes requires invalidating or flushing the MMU's Translation Lookaside Buffer (TLB). Subsequent memory access instructions by the new process experience cold TLB misses, forcing expensive multi-level page table traversals in physical RAM.
- **3. CPU Cache Pollution (Cold-Cache Penalty):**

- The newly scheduled process P_1 finds that the CPU's high-speed L1, L2, and L3 data and instruction caches are filled with cache lines belonging to the previous process P_0 . This causes severe cache misses and memory stall cycles immediately following every process context switch until the working set is reloaded.
- **Optimization Strategies:**
 - **Thread-Level Switching:** Switching between threads of the **same** process avoids switching memory address spaces and flushing the TLB, drastically reducing overhead.
 - **Hardware Support:** Architectures featuring multiple physical register sets (e.g., Sun UltraSPARC) switch contexts simply by changing a hardware register pointer.

Source: Silberschatz et al., Ch 3, p. 115-120 | Source: Tanenbaum, Ch 2, p. 95-110

Topic 2.3: Processes vs. Threads, Thread Types & Thread Lifecycle

GROUP:

Consolidated Questions Covered:

- **Q2:** Analyze differences between User-Level Threads (ULT) and Kernel-Level Threads (KLT).
- **Q3:** What is the difference between process and thread?
- **Q12:** Explain thread life cycle with state diagram.
- **Q14:** List the types of threads and differentiate them.

1. Parameter-Based Comparison: Process (Heavyweight) vs. Thread (Lightweight):

Parameter	Process (Heavyweight Unit)	Thread (Lightweight Unit)
Definition	An independent program in execution with private virtual address space.	The smallest dispatchable unit of CPU execution within a parent process.
Address Space & Memory	Has isolated virtual address space (private Text, Data, Heap, Stack).	Threads within a process share the parent's address space, code, data, and heap.
Private State	Owns complete PCB, file descriptors, memory mappings, and signals.	Owns private Thread ID (TID), Program Counter, Register set, and Stack.
Creation Overhead	High overhead; requires allocating new page tables, PCB, and file tables.	Low overhead; shares existing parent resources; allocates only a small stack.
Context Switch Time	Slow; involves flushing TLB, switching page tables, and saving full context.	Fast; address space remains identical; only registers and stack pointer switched.
Inter-Communication	Requires formal IPC mechanisms (Shared Memory, Message Queues, Pipes).	Fast and direct communication via shared global variables and heap memory.
Fault Isolation	High; a memory fault in one process is isolated and does not affect others.	Low; an illegal memory access in one thread crashes the entire parent process.
Resource Allocation	OS allocates memory, I/O channels, and open files at process level.	Threads share all resources allocated to their parent process.

2. User-Level Threads (ULT) vs. Kernel-Level Threads (KLT):

Parameter	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management & Control	Managed entirely in user space by a user-level thread library (e.g., GNU Pth).	Managed and scheduled directly by the OS Kernel via system calls.
Kernel Awareness	The kernel is completely unaware of individual threads; sees only 1 process.	The kernel maintains individual Thread Control Blocks (TCBs).

Parameter	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Switching Speed	Extremely fast; thread switching requires no kernel mode switch or trap.	Slower; switching requires trapping into kernel mode via interrupt.
Blocking System Calls	If one ULT issues a blocking system call, the entire process blocks .	If one KLT blocks, the kernel schedules another ready thread of that process.
Multicore Parallelism	Cannot achieve true multicore parallelism; all threads share 1 CPU core.	Kernel can schedule multiple threads onto different CPU cores simultaneously.
OS Portability	Highly portable; can run on any OS supporting the user thread library.	OS-dependent; relies on specific kernel multithreading APIs (e.g., Linux NPTL).

3. Multithreading Models:

- **Many-to-One Model:** Multiple user-level threads mapped to a single kernel thread. Fast management, but suffers from entire process blocking on I/O and zero multicore utilization.
- **One-to-One Model (Modern Standard):** Each user thread maps to a dedicated kernel thread (used in Linux, Windows). Provides true multicore concurrency and non-blocking operation, though thread creation carries kernel overhead.
- **Many-to-Many Model:** Multiplexes M user threads onto N kernel threads ($M \geq N$). Optimal flexibility, but complex scheduler implementation.

4. Thread Life Cycle & State Transitions: A thread transitions through five lifecycle states:

1. **New (Born):** The thread instance is created (e.g., `pthread_create()`) and its private execution stack is allocated.
2. **Runnable / Ready:** The thread is ready to execute and waits in the thread run queue for CPU scheduling dispatch.
3. **Running:** The thread's instructions are executing on a physical CPU core.
4. **Blocked / Waiting:** The thread is suspended waiting for an event, I/O completion, or acquiring a synchronization lock (e.g., `pthread_mutex_lock()`).
5. **Dead / Terminated:** The thread finishes execution (via `pthread_exit()`) or is canceled; its resources are cleaned up when joined (`pthread_join()`).

Source: Silberschatz et al., Ch 4, p. 160-185 | Source: Stallings, Ch 4, p. 155-180

Topic 2.4: POSIX Pthread Implementation in C

GROUP:

Consolidated Questions Covered:

- **Q6:** Write and explain a basic pthread creation program in C.

1. Complete SPPU Model C Program Utilizing POSIX Pthreads (`pthread.h`):

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// Structure to pass multiple arguments to worker thread
typedef struct {
    int thread_num;
    int limit;
} ThreadData;

// Worker thread function executed concurrently
void* compute_sum(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    long long sum = 0;

    printf("[Worker Thread %d] Started. Computing sum of 1 to %d...\n",
        data->thread_num, data->limit);

    for (int i = 1; i <= data->limit; i++) {
        sum += i;
    }

    printf("[Worker Thread %d] Completed. Sum = %lld\n", data->thread_num, sum);

    // Allocate heap memory to return result back to main thread
    long long* result = malloc(sizeof(long long));
    *result = sum;
    pthread_exit((void*)result);
}

int main() {
    pthread_t worker_thread;
    ThreadData t_data;
    void* thread_return_val;

    t_data.thread_num = 1;
    t_data.limit = 100;

    printf("[Main Process] Spawning POSIX worker thread using pthread_create()...\n");

    // 1. pthread_create: Spawns new thread
    int status = pthread_create(&worker_thread, NULL, compute_sum, (void*)&t_data);
    if (status != 0) {
        perror("Error: Failed to create thread");
        exit(EXIT_FAILURE);
    }

    printf("[Main Process] Waiting for worker thread to complete execution via pthread_join()...\n");

    // 2. pthread_join: Suspends main thread until worker_thread finishes
    pthread_join(worker_thread, &thread_return_val);

    long long final_sum = *(long long*)thread_return_val;
    printf("[Main Process] Joined successfully. Returned Result from Worker = %lld\n", final_sum);

    free(thread_return_val); // Clean up dynamically allocated return memory
    printf("[Main Process] Main execution finished. Exiting.\n");
    return 0;
}

```

2. Detailed Technical Breakdown of POSIX Pthread Primitives:

- `#include <pthread.h>`: Standard header defining POSIX thread functions, data types, mutexes, and conditional variables. Must compile with `-pthread` flag in GCC (`gcc -pthread prog.c -o prog`).

- `pthread_t worker_thread`: Opaque data type representing the unique thread handle/identifier maintained by the runtime.
- `pthread_create(&tid, attr, start_routine, arg)`:
 - **Parameter 1 (&worker_thread)**: Pointer to `pthread_t` variable storing the created thread ID.
 - **Parameter 2 (NULL)**: Pointer to thread attributes structure (`pthread_attr_t`). Passing NULL uses default attributes (joinable, default stack size).
 - **Parameter 3 (compute_sum)**: Function pointer to the routine to be executed concurrently. Must have the signature `void* func(void* arg)`.
 - **Parameter 4 ((void*)&t_data)**: Pointer to data arguments passed to the worker routine.
- `pthread_join(worker_thread, &thread_return_val)`:
 - Synchronizes execution by suspending the calling thread until the target thread terminates.
 - Reclaims system resources allocated to the finished thread, preventing zombie thread leaks.
 - Captures the return value passed by `pthread_exit()`.
- `pthread_exit((void*)result)`:
 - Explicitly terminates the calling thread without terminating the whole parent process.

Source: Silberschatz et al., Ch 4, p. 165-175 | Source: Tanenbaum, Ch 2, p. 105-120

Topic 2.5: CPU Scheduling Principles (Preemptive vs. Non-Preemptive, Priority Scheduling)

GROUP:

Consolidated Questions Covered:

- **Q8**: Compare preemptive and non-preemptive scheduling with examples.
- **Q13**: Describe priority scheduling with example.

1. CPU Scheduling Fundamentals & The CPU-I/O Burst Cycle: Process execution consists of a cycle of alternating **CPU Bursts** (intense computation) and **I/O Bursts** (waiting for I/O data). The **CPU Scheduler (Short-Term Scheduler)** selects a process from the Ready Queue whenever the CPU becomes idle, and the **Dispatcher** performs the actual context switch.

2. The Four CPU Scheduling Decision Points: CPU scheduling decisions occur under four specific process state transitions:

1. When a process transitions from **Running to Waiting** state (e.g., issuing an I/O request) $\Rightarrow$ **Non-Preemptive**.
2. When a process transitions from **Running to Ready** state (e.g., timer interrupt or quantum expiry) $\Rightarrow$ **Preemptive**.
3. When a process transitions from **Waiting to Ready** state (e.g., I/O completion) $\Rightarrow$ **Preemptive**.
4. When a process **Terminates** $\Rightarrow$ **Non-Preemptive**.

3. Parameter-Based Comparison: Preemptive vs. Non-Preemptive Scheduling:

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Basic Principle	The OS can forcibly interrupt and take the CPU away from a currently running process.	Once a process is allocated the CPU, it holds it until it completes execution or requests I/O.
Context Switch Frequency	High; frequent context switches triggered by timer interrupts and priority changes.	Low; switches occur only upon process completion or voluntary blocking.
Overhead & Complexity	High computational overhead and complex race-condition management on shared kernel data.	Low overhead; simple to implement and manage.
Responsiveness	Very high responsiveness; ideal for multi-user, interactive time-sharing systems.	Poor responsiveness; short jobs get stuck waiting behind long CPU-bound processes (Convoy Effect).

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Starvation Risk	Low-priority processes can suffer starvation if higher-priority processes arrive continuously.	No starvation; every arrived process eventually receives the CPU in arrival order.
Hardware Requirement	Requires dedicated hardware timer interrupt support.	Can operate without a hardware timer.
Representative Algorithms	Round Robin (RR), Shortest Remaining Time First (SRTF), Preemptive Priority.	First-Come First-Served (FCFS), Non-Preemptive Shortest Job First (SJF).

4. Priority Scheduling & The Starvation Problem:

- **Algorithm Principle:** Each process is assigned an integer priority score. The CPU is allocated to the ready process with the highest priority (in standard SPPU conventions, **smaller integer = higher priority**).
- **Starvation (Indefinite Blocking) Hazard:** In a heavily loaded system, a continuous stream of high-priority processes can monopolize the CPU, causing low-priority processes to wait indefinitely (starve) in the Ready Queue.
- **The Aging Solution: Aging** is an algorithmic technique that gradually increases the priority of processes that wait in the system for long durations (e.g., boosting a process's priority by 1 every 10 minutes). This guarantees that every process eventually attains the highest priority and executes.
- **Priority Inversion & Priority Inheritance:** If a low-priority process P_L holds a mutex lock needed by a high-priority process P_H , and a medium process P_M preempts P_L , P_H is indirectly blocked by P_M . Solved via the **Priority Inheritance Protocol**, where P_L temporarily inherits P_H 's high priority until it releases the shared lock.

Source: Silberschatz et al., Ch 5, p. 200-225 | Source: Stallings, Ch 9, p. 390-420

Topic 2.6: Numerical Solved Examples — SJF & Preemptive Priority Scheduling

GROUP:

Consolidated Questions Covered:

- **Q9:** Numerical: Apply Non-Preemptive Shortest Job First (SJF). Calculate average waiting time and turnaround time with Gantt Chart.
- **Q10:** Numerical: Apply Preemptive Priority Scheduling (Lower number = Higher priority). Calculate average waiting time and turnaround time with Gantt Chart.

INTUITION:

Standard SPPU Problem-Solving Formulas:

- **Completion Time (CT):** The exact time instant when a process finishes its final instruction.
- **Turnaround Time (TAT):** Total elapsed time spent in the system:

$$\text{TAT} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$$

- **Waiting Time (WT):** Total time spent waiting in the ready queue before execution:

$$\text{WT} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$$

- **Sanity Check Rule:** For any valid non-idle scheduling execution: $\text{WT} \geq 0$ and $\text{Avg WT} \leq \text{Avg TAT}$.

Problem 1 (Q9): Non-Preemptive Shortest Job First (SJF)

Given Process Dataset:

Process	Arrival Time (AT)	Burst Time (BT)
P_1	0	3
P_2	2	6
P_3	4	4

Process	Arrival Time (AT)	Burst Time (BT)
P_4	5	2

Step-by-Step Chronological Execution Trace:

- **Time $t = 0$:** Only P_1 has arrived ($AT = 0, BT = 3$). Ready Queue contains $\{P_1\}$. P_1 is scheduled and runs **non-preemptively** from $t = 0$ to $t = 3$. Completion Time $CT(P_1) = 3$.
- **Time $t = 3$:** P_1 finishes. At $t = 3$, among remaining processes, only P_2 has arrived ($AT = 2 \leq 3$). Ready Queue contains $\{P_2\}$. P_2 is scheduled and runs non-preemptively from $t = 3$ to $t = 3 + 6 = 9$. Completion Time $CT(P_2) = 9$.
- **During P_2 's execution:** P_3 arrives at $t = 4$ ($BT = 4$), P_4 arrives at $t = 5$ ($BT = 2$).
- **Time $t = 9$:** P_2 finishes. Ready Queue contains $\{P_3(BT = 4), P_4(BT = 2)\}$. Under SJF, P_4 has the shortest burst time ($2 < 4$). P_4 is scheduled and runs from $t = 9$ to $t = 9 + 2 = 11$. Completion Time $CT(P_4) = 11$.
- **Time $t = 11$:** P_4 finishes. Ready Queue contains only $\{P_3(BT = 4)\}$. P_3 runs from $t = 11$ to $t = 11 + 4 = 15$. Completion Time $CT(P_3) = 15$.

Non-Preemptive Shortest Job First (SJF) Gantt Chart**Figure 2.3: Non-Preemptive SJF Timeline Gantt Chart****Textual Exam-Reproducible Gantt Chart:****Complete Result Calculation Table:**

Process	AT	BT	CT	TAT (CT - AT)	WT (TAT - BT)
P_1	0	3	3	$3 - 0 = 3$	$3 - 3 = 0$
P_2	2	6	9	$9 - 2 = 7$	$7 - 6 = 1$
P_3	4	4	15	$15 - 4 = 11$	$11 - 4 = 7$
P_4	5	2	11	$11 - 5 = 6$	$6 - 2 = 4$

Final Mathematical Averages:

- **Average Turnaround Time (Avg TAT):**

$$\text{Avg TAT} = \frac{\sum \text{TAT}}{N} = \frac{3 + 7 + 11 + 6}{4} = \frac{27}{4} = 6.75 \text{ ms}$$

- **Average Waiting Time (Avg WT):**

$$\text{Avg WT} = \frac{\sum \text{WT}}{N} = \frac{0 + 1 + 7 + 4}{4} = \frac{12}{4} = 3.00 \text{ ms}$$

Problem 2 (Q10): Preemptive Priority Scheduling

Given Process Dataset (Lower numerical value = Higher Priority; Priority 1 is highest):

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P_1	0	3	4
P_2	2	6	2
P_3	4	4	3
P_4	5	2	1 (Highest)

Step-by-Step Preemptive Chronological Execution Trace:

- **Time $t = 0$:** Only P_1 arrives (AT = 0, BT = 3, Priority 4). P_1 begins execution.
- **Time $t = 2$:** P_2 arrives (AT = 2, BT = 6, Priority 2). P_1 has executed for 2 ms (remaining $BT(P_1) = 3 - 2 = 1$). Since Priority 2 < 4 (higher priority), P_2 **preempts** P_1 . P_1 is moved to Ready Queue. P_2 begins execution.
- **Time $t = 4$:** P_3 arrives (AT = 4, BT = 4, Priority 3). Currently running is P_2 (Priority 2). Since Priority 2 < 3, P_2 has higher priority and continues execution. P_3 is added to Ready Queue.
- **Time $t = 5$:** P_4 arrives (AT = 5, BT = 2, Priority 1). P_2 has executed for 3 ms from $t = 2$ to 5 (remaining $BT(P_2) = 6 - 3 = 3$). Since Priority 1 < 2 (highest priority), P_4 **preempts** P_2 . P_2 is moved to Ready Queue. P_4 begins execution.
- **Time $t = 5$ to 7:** P_4 executes for its full 2 ms burst time. Completes at $t = 7$. Completion Time $CT(P_4) = 7$.
- **Time $t = 7$:** P_4 terminates. Ready Queue contains: $\{P_2(\text{rem BT} = 3, \text{Pri} = 2), P_3(\text{BT} = 4, \text{Pri} = 3), P_1(\text{rem BT} = 1, \text{Pri} = 4)\}$. Highest priority is P_2 (Priority 2). P_2 resumes and executes for 3 ms until $t = 7 + 3 = 10$. Completes at $t = 10$. Completion Time $CT(P_2) = 10$.
- **Time $t = 10$:** P_2 terminates. Ready Queue contains: $\{P_3(\text{BT} = 4, \text{Pri} = 3), P_1(\text{rem BT} = 1, \text{Pri} = 4)\}$. Highest priority is P_3 (Priority 3). P_3 executes for 4 ms until $t = 10 + 4 = 14$. Completes at $t = 14$. Completion Time $CT(P_3) = 14$.
- **Time $t = 14$:** P_3 terminates. Only P_1 remains (rem BT = 1, Pri = 4). P_1 resumes and executes for 1 ms until $t = 14 + 1 = 15$. Completes at $t = 15$. Completion Time $CT(P_1) = 15$.

Preemptive Priority Scheduling Gantt Chart (Lower Number = Higher Priority)

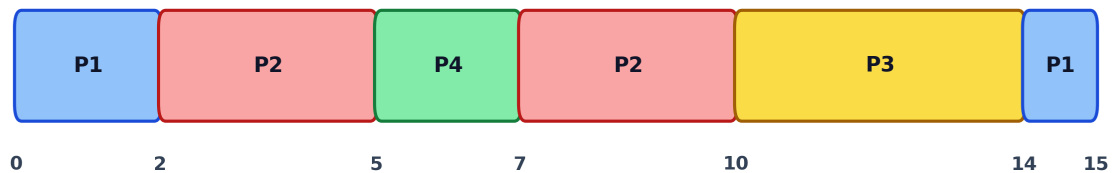


Figure 2.4: Preemptive Priority Scheduling Gantt Chart

Textual Exam-Reproducible Gantt Chart:



Complete Result Calculation Table:

Process	AT	BT	Pri	CT	TAT (CT - AT)	WT (TAT - BT)
P_1	0	3	4	15	$15 - 0 = 15$	$15 - 3 = 12$
P_2	2	6	2	10	$10 - 2 = 8$	$8 - 6 = 2$
P_3	4	4	3	14	$14 - 4 = 10$	$10 - 4 = 6$
P_4	5	2	1	7	$7 - 5 = 2$	$2 - 2 = 0$

Final Mathematical Averages:**• Average Turnaround Time (Avg TAT):**

$$\text{Avg TAT} = \frac{\sum \text{TAT}}{N} = \frac{15 + 8 + 10 + 2}{4} = \frac{35}{4} = 8.75 \text{ ms}$$

• Average Waiting Time (Avg WT):

$$\text{Avg WT} = \frac{\sum \text{WT}}{N} = \frac{12 + 2 + 6 + 0}{4} = \frac{20}{4} = 5.00 \text{ ms}$$

Source: Silberschatz et al., Ch 5, p. 205-230 | Source: Stallings, Ch 9, p. 400-425